

Revenue Documentation

1 Introduction

The `revenue.py` script calculates the rate-based revenue requirement from a utility perspective. Revenue is defined as Capital Costs PV (build + ROW + environmental mitigation) times an allowed return rate, computed annually over the project lifetime and discounted to present value. The result is written to the batch summary (CSV or JSON) and used in BCR and reporting. Revenue calculation can be disabled via `financial.revenue.rate_based.enabled` in the financing config.

2 Method Overview

2.1 Purpose

1. Load rate-based revenue parameters from financing data: `enabled` and `allowed_return_rate` (from `financial.revenue.rate_based`).
2. If not enabled: write zero revenue to the output manager and return.
3. Load project technical details (`delay_years`, `construction_years`, `project_lifetime`) and financing details (`WACC`, `base_year`).
4. Get capital costs PV from `batch_summary.csv` (CSV mode) or from `json_output_<id>_build_costs_row_costs.json`, `environmental_mitigation.json` (JSON mode); `capital = build_cost_pv + row_cost_pv + env_mitigation_pv`.
5. Compute `annual_revenue = capital_costs_pv × allowed_return_rate`; `total_nominal = annual × project_lifetime`.
6. Compute present value of revenue stream using `calculate_present_value` (real WACC, `project_lifetime`, `start_year = COD` from `delay` and `construction` years).
7. Write `revenue_nominal`, `revenue_pv`, `annual_revenue`, `rate_base_pv`, `allowed_return_rate` to the output manager and call `write_batch_summary()`.

2.2 Calculation Approach

Rate base is the present value of capital costs (build + ROW + environmental mitigation). Annual revenue requirement (utility perspective) is rate base times the allowed return rate. Revenue is assumed to start at commercial operation date (COD = delay_years + construction_years) and continue for project_lifetime years. Present value uses real WACC and the same discounting as other CTCC PV calculations.

$$\text{Rate Base (PV)} = \text{build_cost_pv} + \text{row_cost_pv} + \text{env_mitigation_pv} \quad (1)$$

$$\text{Annual Revenue} = \text{Rate Base} \times \text{allowed_return_rate} \quad (2)$$

$$\text{Total Revenue (nominal)} = \text{Annual Revenue} \times \text{project_lifetime} \quad (3)$$

$$\text{Revenue (PV)} = \sum_{t=\text{COD}}^{\text{COD}+\text{project_lifetime}-1} \frac{\text{Annual Revenue}}{(1+r)^t} \quad (4)$$

where r is the real WACC and $\text{COD} = \text{delay_years} + \text{construction_years}$.

2.3 Inputs and Outputs

Inputs:

- Financing data (YAML or JSON): `financial.revenue.rate_based.enabled`, `financial.revenue.allowed_return_rate`, `financial.wacc_nominal`, `inflation_rate`, `base_year` for PV.
- Project technical details: `delay_years`, `construction_years`, `project_lifetime`.
- Capital costs PV: from `batch_summary.csv` (CSV mode) or from `json_output_<scenario_id>_build_row_costs.json`, `environmental_mitigation.json` (JSON mode); requires `CTCC_SCENARIO_ID` in JSON mode.

Outputs:

- `revenue_nominal`, `revenue_pv`, `annual_revenue`, `rate_base_pv`, `allowed_return_rate` passed to `CTCCOutputManager.add_revenue(results)`; then `write_batch_summary()`. Used by BCR calculator (benefits side) and in final JSON/CSV summary.

2.4 Integration with Other Scripts

- `ctcc.py` — Runs `revenue.py` after `weighted_miles`, `build_costs`, `row_costs`, `environmental_mitigation` so capital costs are available.
- `smart_loaders.py` — `load_financing_details`, `get_financing_data_raw`, `load_project_technical_details` (centralized).
- `financial_utils.py` — `calculate_present_value`, `calculate_cod_year`.
- `smart_output.py` — `CTCCOutputManager` for `add_revenue` and `write_batch_summary`.

- `path_config.py` — `OUTPUTS_DIR` for `batch_summary.csv` and JSON output paths.
- `bcr_calculator.py` — Reads revenue (e.g. `revenue_pv`, `revenue_nominal`) from batch summary or aggregated JSON for BCR benefits.

3 Key Functions

3.1 `load_rate_based_revenue_parameters()`

Loads `enabled` and `allowed_return_rate` from `get_financing_data_raw()["financial"]["revenue"]`. Defaults: `enabled=False`, `allowed_return_rate=0.10`.

Returns: tuple: (`enabled: bool`, `allowed_return_rate: float`).

3.2 `load_project_technical_details()`

Uses centralized `load_project_technical_details` and returns (`delay_years`, `construction_years`, `project_lifetime`) from the project technical details dataclass.

3.3 `get_capital_costs_pv()`

Returns `build_cost_pv + row_cost_pv + env_mitigation_pv`. In JSON mode (when `CTCC_OUTPUT_MODE=json` and `CTCC_SCENARIO_ID` is set), reads from `json_output<scenario_id>_build_costs.json`, `row_costs.json`, `environmental_mitigation.json` in `OUTPUTS_DIR`. In CSV mode, reads from `batch_summary.csv` (row matching `scenario_id`, or last row if `scenario_id` not set). Returns 0 if files missing or scenario not found.

Key logic (annual revenue and PV):

```

1 annual_revenue = capital_costs_pv * allowed_return_rate
2 total_revenue_nominal = annual_revenue * project_lifetime
3 revenue_pv = calculate_present_value(
4     annual_revenue,
5     financing.wacc_real,
6     project_lifetime,
7     start_year=calculate_cod_year(delay_years, construction_years),
8 )

```

3.4 `main()`

Checks `load_rate_based_revenue_parameters()`; if not enabled, writes zeros via `add_revenue` and `write_batch_summary` and returns. Otherwise loads details, gets `get_capital_costs_pv()`, computes annual revenue, nominal total, and PV, then adds results and writes batch summary.

4 Example

With `financial.revenue.rate_based.enabled = true` and `allowed_return_rate = 0.10`, and capital costs $PV = \$500M$, `project_lifetime = 40`, `COD = 5`:

- Annual revenue = $500 \times 0.10 = \$50M$.
- Total revenue (nominal) = $50 \times 40 = \$2000M$.
- Revenue PV = present value of \$50M per year for 40 years starting at year 5, discounted at real WACC.

Standalone run (after capital scripts have run):

```
1 cd scripts
2 python revenue.py
```